SCIKEY DATABASE — Critical Query Execution Plans

Oracle 19c | PDB: pdb | Schema: SCIKEY *Last Updated: April 2026*

Table of Contents

SCIKEY DATABASE — Critical Query Execution Plans **1**
- Table of Contents 1
- Overview 2
- How to Read an Oracle Execution Plan 3
- Q4 — Keyword Filter, Narrow Projection 4
  - Plan 4
  - How It Executes 5
  - Key Observations 5
  - Watch For 5
- Q7 — Keyword Filter, Wide Projection 6
  - Plan 6
  - How It Executes 7
  - Key Observations 7
  - Watch For 8
- Q6 — MV-to-MV Join on author_key 8
  - Plan 8
  - How It Executes 9
  - Key Observations 10
  - Watch For 10
- Q9 — Organism Aggregation, No Filter 10
  - Plan 11
  - How It Executes 12
  - Key Observations 12
  - Watch For 13
- Q10 — CLOB Access on Base Tables, Keyword Filter 13
  - Plan 14
  - How It Executes 15

Overview

This document covers the verified execution plans for the five queries identified as most critical to the SCIKEY workload. Plans were captured using `DBMS_XPLAN.DISPLAY` with `ALL +PARTITION +PROJECTION` format against live statistics.

The five queries are:

ID	Query	Source Object(s)	Cost
Q4	Keyword filter, narrow projection	MV_DOC_AUTHOR_KEYWORD	6
Q7	Keyword filter, wide projection	MV_DOC_AUTHOR_KEYWORD	6
Q6	MV-to-MV join on author_key	MV_AUTHOR_ORGANISM_DISTINCT + MV_DOC_AUTHOR_KEYWORD	14502
Q9	Organism aggregation, no filter	MV_AUTHOR_ORGANISM_DISTINCT	12723
Q10	CLOB access on base tables, keyword filter	DOCUMENT + DOC_KEYWORD + KEYWORD	42

Q4 and Q7 are both documented because keyword_text is expected to be the most frequently queried column in the system. Although they share the same plan hash and index access path, their projection widths differ — Q7 fetches additional columns from the MV row, which affects how much data is read per ROWID lookup at step 1.

How to Read an Oracle Execution Plan

*Oracle's step numbers are reference identifiers, not an execution sequence. Reading top to bottom is misleading — step 0 (SELECT STATEMENT) is always listed first but executes last. The actual execution order follows the **indentation of the plan tree**, not the step numbers.*

The rules are:

1. ***Find the deepest, most-indented operation — that is always where execution begins.*** *It is the first row source to produce data.*
2. ***Work outward (up the indentation level).*** *Each parent operation waits for its children to produce rows before it can execute.*
3. ***When two operations share the same indentation level under the same parent, the upper one executes first.*** *In a hash join this means the upper child is the build side (loaded into the hash table), and the lower child is the probe side (streamed through it).*
4. ***Step 0 always executes last*** *— it is the final SELECT STATEMENT that returns the result to the client.*

For Q10 as a concrete example, the plan tree looks like this when indentation is used to show the hierarchy:

```
0  SELECT STATEMENT

1    NESTED LOOPS                    ← executes 9th, joins result of 2+3 to step 11

2      NESTED LOOPS                  ← executes 7th, joins result of 3 to step 10

3        NESTED LOOPS                ← executes 5th, joins keyword rows to doc_keyword rows

4          PARTITION RANGE SINGLE      ← executes 1st, identifies partition 25

5            TABLE ACCESS KEYWORD      ← executes 2nd, fetches keyword row

6              INDEX RANGE SCAN        ← executes 1st (innermost), scans
IDX_KEYWORD_TEXT

7          PARTITION REFERENCE ALL     ← executes 3rd, sets up doc_keyword partitions

8            TABLE ACCESS DOC_KEYWORD  ← executes 4th, fetches doc_keyword rows

9              INDEX RANGE SCAN        ← executes 3rd, scans IDX_DK_KEYWORD

10       INDEX UNIQUE SCAN PK_DOCUMENT  ← executes 6th, looks up document ROWID

11     TABLE ACCESS DOCUMENT           ← executes 8th, fetches document row + CLOB
```

Actual execution order: ***6 → 5 → 4 → 9 → 8 → 7 → 3 → 10 → 2 → 11 → 1 → 0***

Each query section below documents steps in this true execution order rather than by step number, so the narrative follows what Oracle actually does at runtime.

Q4 — Keyword Filter, Narrow Projection

SELECT title, keyword_text

FROM SCIKEY.MV_DOC_AUTHOR_KEYWORD

WHERE keyword_text LIKE 'algorithm%';

Plan

Plan hash value: 2606838996

```
| Id | Operation                          | Name                    | Rows | Bytes | Cost | Time     |

|----|------------------------------------|-------------------------|------|-------|------|----------|

|  0 | SELECT STATEMENT                   |                         |    5 |   570 |    6 | 00:00:01 |

|  1 |  MAT_VIEW ACCESS BY INDEX ROWID BATCHED | MV_DOC_AUTHOR_KEYWORD   |
5 |   570 |    6 | 00:00:01 |

|* 2 |   INDEX RANGE SCAN                 | IDX_MV_DAK_KEYWORD_TEXT |    5 |       |    3 |
00:00:01 |
```

Predicate Information:

```
  2 - access("KEYWORD_TEXT" LIKE 'algorithm%')

      filter("KEYWORD_TEXT" LIKE 'algorithm%')
```

Projection:

```
  1 - "TITLE"[VARCHAR2,600], "KEYWORD_TEXT"[VARCHAR2,1000]
```

2 - "MV_DOC_AUTHOR_KEYWORD".ROWID[ROWID,10], "KEYWORD_TEXT"[VARCHAR2,1000]

How It Executes

Execution order: ***step 2 → step 1 → step 0***

Step 2 (executes first) — INDEX RANGE SCAN on IDX_MV_DAK_KEYWORD_TEXT: *This is the innermost, most-indented operation, so it runs first. Oracle uses the prefix* `algorithm` *as the access predicate to range-scan the index. Because the LIKE pattern has a fixed prefix with no leading wildcard, the B-tree can seek directly to the first matching leaf block and scan forward. The filter predicate at this step handles the trailing* `%` *portion. Only ROWIDs and* `keyword_text` *values are returned from the index — no table access yet.*

Step 1 (executes second) — MAT_VIEW ACCESS BY INDEX ROWID BATCHED: *With ROWIDs in hand from step 2, Oracle now fetches the 5 estimated MV rows. The BATCHED variant groups multiple ROWID lookups into a single I/O call (table prefetch), reducing round-trips. At this step* `title` *is retrieved alongside* `keyword_text`*. Since both columns are in the MV and the MV is indexed on* `keyword_text`*, this is a fully self-contained access path — the base tables are never touched.*

Step 0 (executes last) — SELECT STATEMENT: *Returns the assembled result to the client.*

Key Observations

- ***Total cost: 6.*** *This is exceptionally low. The MV eliminates the multi-table join that would otherwise be required.*
- ***Estimated rows: 5.*** *Statistics expect the prefix* `algorithm%` *to match a small, selective slice of the keyword population. This estimate will shift for broader prefixes like* `data%` *or* `model%` *— verify with those prefixes if response time varies noticeably.*
- ***Both access and filter predicates appear at step 2.*** *This is expected and correct for LIKE with a fixed prefix. The access predicate narrows the index range; the filter predicate re-evaluates each qualifying entry against the full pattern. No redundant work occurs here.*
- ***No partition information in the plan.*** *The MV itself is not partitioned, so partition columns are absent. Partitioning benefits for keyword lookups are realised on the base* `keyword` *table, which Q10 exercises.*

Watch For

If `keyword_text` *cardinality grows significantly after a bulk keyword import, the row estimate of 5 may become stale. Re-gather MV statistics after large refreshes:*

DBMS_STATS.GATHER_TABLE_STATS(ownname => 'SCIKEY', tabname => 'MV_DOC_AUTHOR_KEYWORD',

estimate_percent => DBMS_STATS.AUTO_SAMPLE_SIZE, method_opt => 'FOR ALL COLUMNS SIZE AUTO');

Q7 — Keyword Filter, Wide Projection

SELECT title, last_name, keyword_text, doi_id

FROM SCIKEY.MV_DOC_AUTHOR_KEYWORD

WHERE keyword_text LIKE 'algorithm%';

Plan

Plan hash value: 2606838996

```
| Id | Operation                          | Name                    | Rows | Bytes | Cost | Time     |

|----|------------------------------------|-------------------------|------|-------|------|----------|

|  0 | SELECT STATEMENT                   |                         |    5 |   685 |    6 | 00:00:01 |

|  1 |  MAT_VIEW ACCESS BY INDEX ROWID BATCHED | MV_DOC_AUTHOR_KEYWORD   |
5 |   685 |    6 | 00:00:01 |

|* 2 |   INDEX RANGE SCAN                 | IDX_MV_DAK_KEYWORD_TEXT |    5 |       |    3 |
00:00:01 |
```

Predicate Information:

2 - access("KEYWORD_TEXT" LIKE 'algorithm%')

filter("KEYWORD_TEXT" LIKE 'algorithm%')

Projection:

1 - "TITLE"[VARCHAR2,600], "DOI_ID"[VARCHAR2,255], "LAST_NAME"[VARCHAR2,255],

"KEYWORD_TEXT"[VARCHAR2,1000]

2 - "MV_DOC_AUTHOR_KEYWORD".ROWID[ROWID,10], "KEYWORD_TEXT"[VARCHAR2,1000]

How It Executes

The plan structure is identical to Q4 — same plan hash value (2606838996), same two-step index range scan followed by batched ROWID lookup. The difference is visible only in the projection at step 1: Q7 pulls four columns (`title`, `doi_id`, `last_name`, `keyword_text`) versus Q4's two.

Execution order: ***step 2*** *→* ***step 1*** *→* ***step 0*** *— identical structure to Q4.*

Step 2 (executes first) — INDEX RANGE SCAN on IDX_MV_DAK_KEYWORD_TEXT: *Identical to Q4. The index returns only `keyword_text` and the ROWID at this step — the extra columns requested in the SELECT are not index-resident and do not affect this step.*

Step 1 (executes second) — MAT_VIEW ACCESS BY INDEX ROWID BATCHED: *The MV row is fetched for each ROWID collected at step 2. Because Q7 requests `last_name` and `doi_id` in addition to `title` and `keyword_text`, Oracle reads a wider slice of each MV row. The byte estimate increases from 570 to 685 per the plan output, reflecting this. The cost remains 6 because the number of I/O operations is unchanged — the extra columns are on the same data block.*

Step 0 (executes last) — SELECT STATEMENT: *Returns the assembled result to the client.*

Key Observations

- ***Same plan hash as Q4.*** *Oracle recognises these as structurally identical access paths. The plan cache will hold one plan shape for this MV + index combination regardless of which columns are selected, provided the WHERE clause matches.*
- ***Byte estimate difference (570 vs 685).*** *The 115-byte increase comes from adding `last_name` (VARCHAR2,255) and `doi_id` (VARCHAR2,255) to the projection. Under low selectivity this is negligible, but at scale — for prefixes matching thousands of rows — the wider row fetch will consume proportionally more buffer cache and network bandwidth.*

- ***Cost identical at 6.*** *Both queries are I/O-bound by the same index scan and ROWID lookups. The wider projection does not add a cost unit here because Oracle's costing model for single-table MV access does not penalise column width at this cardinality.*

Watch For

Q4 and Q7 are the two highest-frequency query patterns in the system given the expected volume of keyword lookups. For broad prefixes (`data%`, `model%`, `system%`) the row estimate of 5 will be severely underestimated — the `data%` partition alone holds 45,564 keywords. If execution times diverge between narrow and wide projections at high row counts, consider whether `doi_id` and `last_name` could be deferred to a secondary lookup rather than fetched in the initial keyword scan.

Q6 — MV-to-MV Join on author_key

SELECT aod.last_name, aod.struct_name, mv.title

FROM SCIKEY.MV_AUTHOR_ORGANISM_DISTINCT aod

JOIN SCIKEY.MV_DOC_AUTHOR_KEYWORD mv ON aod.author_key = mv.author_key

WHERE aod.last_name LIKE 'S%';

Plan

Plan hash value: 2266732143

```
| Id | Operation            | Name                        | Rows  | Bytes  | Cost  | Time     |
|----|----------------------|-----------------------------|-------|--------|-------|----------|
|  0 | SELECT STATEMENT     |                             | 1134K | 174M   | 14502 | 00:00:02 |
|* 1 |  HASH JOIN           |                             | 1134K | 174M   | 14502 | 00:00:02 |
|* 2 |   MAT_VIEW ACCESS FULL | MV_AUTHOR_ORGANISM_DISTINCT | 24419 | 1406K  | 1016 | 00:00:01 |
```

| 3 | MAT_VIEW ACCESS FULL | MV_DOC_AUTHOR_KEYWORD | 2422K | 235M | 13476 | 00:00:02 |

Predicate Information:

1 - access("AOD"."AUTHOR_KEY"="MV"."AUTHOR_KEY")

2 - filter("AOD"."LAST_NAME" LIKE 'S%')

Projection:

1 - "AOD"."LAST_NAME"[VARCHAR2,255], "AOD"."STRUCT_NAME"[VARCHAR2,255], "MV"."TITLE"[VARCHAR2,600]

2 - "AOD"."AUTHOR_KEY"[NUMBER,22], "AOD"."LAST_NAME"[VARCHAR2,255], "AOD"."STRUCT_NAME"[VARCHAR2,255]

3 - "MV"."TITLE"[VARCHAR2,600], "MV"."AUTHOR_KEY"[NUMBER,22]

Note: this is an adaptive plan

How It Executes

Execution order: ***step 2 → step 3 → step 1 → step 0***

At the same indentation level under the HASH JOIN at step 1, step 2 appears above step 3 — so step 2 (the build side) executes first, then step 3 (the probe side) streams through the hash table.

Step 2 (executes first) — MAT_VIEW ACCESS FULL on MV_AUTHOR_ORGANISM_DISTINCT (build side): *Oracle fully scans the smaller MV and applies the* `last_name LIKE 'S%'` *filter inline, reducing an estimated 406K total rows down to 24,419. These filtered rows are loaded into a hash table in memory keyed on* `author_key`*. This MV is chosen as the build side because after filtering it is substantially smaller than the probe side.*

Step 3 (executes second) — MAT_VIEW ACCESS FULL on MV_DOC_AUTHOR_KEYWORD (probe side): *Oracle performs a full scan of the larger MV, estimated at 2.4 million rows, fetching only* `author_key` *and* `title`*. Each row is probed against the hash table built in step 2. Rows with a matching* `author_key` *are passed up to the join operator.*

Step 1 (executes third) — HASH JOIN: *Assembles the joined rows from the two MV scans above. The join predicate* `aod.author_key = mv.author_key` *is evaluated via hash lookup. The output includes* `last_name`*,* `struct_name`*, and* `title`*.*

Step 0 (executes last) — SELECT STATEMENT: *Returns the result to the client.*

Key Observations

- ***Cost: 14,502.*** *This is the most expensive query in the set by a wide margin, driven almost entirely by the full scan of MV_DOC_AUTHOR_KEYWORD at cost 13,476. This MV is large and unfiltered on the probe side.*
- ***Adaptive plan.*** *Oracle has reserved the right to switch from hash join to nested loops at runtime depending on actual row counts at step 2. If the* `last_name LIKE 'S%'` *filter returns far fewer rows than 24,419 at execution time, Oracle may convert to a nested loop with an index probe into MV_DOC_AUTHOR_KEYWORD. Whether an index on* `author_key` *exists on that MV determines whether this adaptation is beneficial.*
- ***No index used on either MV.*** *The* `last_name` *filter on MV_AUTHOR_ORGANISM_DISTINCT is applied as a post-scan filter, not an access predicate. If an index on* `last_name` *existed on this MV, Oracle could range-scan it and avoid scanning all 406K rows. This is the primary tuning opportunity for Q6.*
- ***Result set: 1.1 million rows estimated.*** *This is a large result set to materialise on the client. In practice, Q6 should always be paired with a FETCH FIRST / ROWNUM limit or additional filter predicates unless the full result is genuinely required.*

Watch For

The full scan of MV_DOC_AUTHOR_KEYWORD (2.4M rows, cost 13,476) will dominate execution time at any data volume. If Q6 frequency increases, consider adding an index on `author_key` *to MV_DOC_AUTHOR_KEYWORD to enable a nested loop against the filtered build side. Also consider whether the* `last_name` *filter could be pushed earlier by adding an index on* `last_name` *to MV_AUTHOR_ORGANISM_DISTINCT.*

Q9 — Organism Aggregation, No Filter

SELECT struct_name, hal_structure_id, COUNT(DISTINCT author_key) AS auth_cnt

FROM SCIKEY.MV_AUTHOR_ORGANISM_DISTINCT

GROUP BY organism_key, struct_name, hal_structure_id

ORDER BY auth_cnt DESC;

Plan

Plan hash value: 2137883071

```
| Id | Operation              | Name                        | Rows  | Bytes | TempSpc | Cost  | Time     |
|----|------------------------|-----------------------------|-------|-------|---------|-------|----------|
|  0 | SELECT STATEMENT       |                             |  406K |  25M  |         | 12723 | 00:00:01 |
|  1 |  SORT ORDER BY         |                             |  406K |  25M  |  31M    | 12723 | 00:00:01 |
|  2 |   HASH GROUP BY        |                             |  406K |  25M  |  31M    | 12723 | 00:00:01 |
|  3 |    VIEW                | VM_NWVW_1                   |  406K |  25M  |         |  4629 | 00:00:01 |
|  4 |     HASH GROUP BY      |                             |  406K |  22M  |  28M    |  4629 | 00:00:01 |
|  5 |      MAT_VIEW ACCESS FULL | MV_AUTHOR_ORGANISM_DISTINCT | 406K |  22M  |
|  1015 | 00:00:01 |
```

Projection:

1 - COUNT("$vm_col_4")[22], "$vm_col_2"[VARCHAR2,255], "$vm_col_1"[NUMBER,22]

2 - "$vm_col_3"[NUMBER,22], "$vm_col_2"[VARCHAR2,255], "$vm_col_1"[NUMBER,22], COUNT("$vm_col_4")[22]

3 - "$vm_col_1"[NUMBER,22], "$vm_col_2"[VARCHAR2,255], "$vm_col_3"[NUMBER,22], "$vm_col_4"[NUMBER,22]

4 - "AUTHOR_KEY"[NUMBER,22], "ORGANISM_KEY"[NUMBER,22], "STRUCT_NAME"[VARCHAR2,255], "HAL_STRUCTURE_ID"[NUMBER,22]

5 - "AUTHOR_KEY"[NUMBER,22], "ORGANISM_KEY"[NUMBER,22],

"STRUCT_NAME"[VARCHAR2,255], "HAL_STRUCTURE_ID"[NUMBER,22]

How It Executes

Execution order: ***step 5 → step 4 → step 3 → step 2 → step 1 → step 0***

Step 5 (executes first) — MAT_VIEW ACCESS FULL on MV_AUTHOR_ORGANISM_DISTINCT: *The innermost operation. A full scan of all 406K rows with no filter predicate — every row is read. This is unavoidable given the query has no WHERE clause.*

Step 4 (executes second) — HASH GROUP BY (inner, feeds VM_NWVW_1): *Oracle performs a first-pass aggregation to handle `COUNT(DISTINCT author_key)`. Because `DISTINCT` inside an aggregate cannot be resolved in a single grouping pass, Oracle rewrites the query internally and introduces the view alias `VM_NWVW_1`. This inner hash group by deduplicates `author_key` values per `organism_key` group before any counting occurs. Spills 28M to temp.*

Step 3 (executes third) — VIEW VM_NWVW_1: *The internal view materialises the deduplicated intermediate result from step 4, making it available to the outer aggregation as if it were a named row source.*

Step 2 (executes fourth) — HASH GROUP BY (outer): *A second grouping pass now computes the final `COUNT` per `(organism_key, struct_name, hal_structure_id)` group using the deduplicated data from step 3. Spills another 31M to temp.*

Step 1 (executes fifth) — SORT ORDER BY: *The aggregated result is sorted descending by `auth_cnt`. At 406K estimated output rows this also spills 31M to temp.*

Step 0 (executes last) — SELECT STATEMENT: *Returns the sorted result to the client.*

Key Observations

- ***Cost: 12,723.*** *High cost driven by the full MV scan, two hash group-by passes, and a sort — all of which spill to temp.*
- ***Two hash group-by passes for COUNT(DISTINCT).*** *Oracle cannot compute `COUNT(DISTINCT)` in a single aggregation pass. The two-phase rewrite via VM_NWVW_1 is the standard execution path and cannot be avoided without restructuring the query (e.g., by pre-aggregating distinct author counts into the MV itself).*

- ***Total temp usage: ~90M.*** *Steps 2, 4, and the sort all declare TempSpc of 28–31M each. Ensure the TEMP tablespace has sufficient headroom, especially if Q9 runs concurrently with other sorting or aggregation workloads.*
- ***No filter, no pruning.*** *This query is a deliberate full-scan aggregate. It is expected to be expensive. If it is run frequently, the correct mitigation is to pre-compute the result into a dedicated summary MV and refresh it on a schedule rather than computing it on demand.*

Watch For

If `COUNT(DISTINCT author_key)` *per organism is needed regularly, create a dedicated summary MV:*

```
CREATE MATERIALIZED VIEW MV_ORGANISM_AUTHOR_COUNTS

BUILD IMMEDIATE REFRESH COMPLETE ON DEMAND AS

SELECT organism_key, struct_name, hal_structure_id,

    COUNT(DISTINCT author_key) AS auth_cnt

FROM MV_AUTHOR_ORGANISM_DISTINCT

GROUP BY organism_key, struct_name, hal_structure_id;
```

This reduces Q9's execution to a single indexed lookup or fast full scan with no GROUP BY or sort overhead at query time.

Q10 — CLOB Access on Base Tables, Keyword Filter

```
SELECT DBMS_LOB.SUBSTR(d.abstract, 200, 1)

FROM SCIKEY.document d

JOIN SCIKEY.doc_keyword dk ON d.document_key = dk.document_key

JOIN SCIKEY.keyword k ON dk.keyword_key = k.keyword_key

WHERE d.abstract IS NOT NULL

  AND k.keyword_text LIKE 'algorithm%';
```

Plan

Plan hash value: 1019474582

```
| Id | Operation                                    | Name             | Rows | Bytes | Cost | Time     | Pstart |
Pstop |

|----|----------------------------------------------|------------------|------|-------|------|----------|--------|-------|

|  0 | SELECT STATEMENT                             |                  |    3 |  3597 |   42 | 00:00:01 |        |
|

|  1 |  NESTED LOOPS                                |                  |    3 |  3597 |   42 | 00:00:01 |        |
|

|  2 |   NESTED LOOPS                               |                  |    3 |  3597 |   42 | 00:00:01 |        |
|

|  3 |    NESTED LOOPS                              |                  |    3 |   123 |   39 | 00:00:01 |        |       |

|  4 |     PARTITION RANGE SINGLE                   |                  |    1 |    31 |    3 | 00:00:01 |     25
|    25 |

|  5 |      TABLE ACCESS BY LOCAL INDEX ROWID BATCHED | KEYWORD          |    1 |    31 |
3 | 00:00:01 |     25 |    25 |

|* 6 |       INDEX RANGE SCAN                       | IDX_KEYWORD_TEXT |    1 |       |    2 |
00:00:01 |     25 |    25 |

|  7 |     PARTITION REFERENCE ALL                  |                  |    3 |    30 |   36 | 00:00:01 |      1
|    34 |

|  8 |      TABLE ACCESS BY LOCAL INDEX ROWID BATCHED | DOC_KEYWORD      |    3 |
30 |   36 | 00:00:01 |      1 |    34 |

|* 9 |       INDEX RANGE SCAN                       | IDX_DK_KEYWORD   |    3 |       |   34 |
00:00:01 |      1 |    34 |

|*10 |    INDEX UNIQUE SCAN                         | PK_DOCUMENT      |    1 |       |    0 | 00:00:01
|       |       |

|*11 |   TABLE ACCESS BY GLOBAL INDEX ROWID         | DOCUMENT         |    1 |  1158 |
1 | 00:00:01 | ROWID  | ROWID |
```

Predicate Information:

6 - access("K"."KEYWORD_TEXT" LIKE 'algorithm%')

filter("K"."KEYWORD_TEXT" LIKE 'algorithm%')

9 - access("DK"."KEYWORD_KEY"="K"."KEYWORD_KEY")

10 - access("D"."DOCUMENT_KEY"="DK"."DOCUMENT_KEY")

11 - filter("D"."ABSTRACT" IS NOT NULL)

Projection:

1 - "D"."ABSTRACT"[LOB,4000]

3 - "K"."KEYWORD_KEY"[NUMBER,22], "DK"."DOCUMENT_KEY"[NUMBER,22]

4 - "K"."KEYWORD_KEY"[NUMBER,22]

5 - "K"."KEYWORD_KEY"[NUMBER,22]

6 - "K".ROWID[ROWID,10]

7 - "DK"."DOCUMENT_KEY"[NUMBER,22]

8 - "DK"."DOCUMENT_KEY"[NUMBER,22]

9 - "DK".ROWID[ROWID,10]

10 - "D".ROWID[ROWID,10]

11 - "D"."ABSTRACT"[LOB,4000]

Note: this is an adaptive plan

How It Executes

Execution order: ***step 6 → step 5 → step 4 → step 9 → step 8 → step 7 → step 3 → step 10 → step 2 → step 11 → step 1 → step 0***

This is the most complex plan in the set. The three nested loop operators (steps 1, 2, 3) each have two children — in each pair, the upper child (higher in the plan listing) executes first as the driving side, and the lower child executes once per row returned by the driver.

Step 6 (executes first) — INDEX RANGE SCAN on IDX_KEYWORD_TEXT (partition 25): *The deepest, most-indented operation. Oracle range-scans the keyword index within the single pruned partition, applying both the access and filter predicates for* `keyword_text LIKE 'algorithm%'`*. Returns ROWIDs for matching keyword rows.*

Step 5 (executes second) — TABLE ACCESS BY LOCAL INDEX ROWID BATCHED on KEYWORD: *Fetches the actual keyword row(s) from partition 25 using the ROWIDs from step 6. Returns* `keyword_key` *for use in the next join.*

Step 4 (executes third) — PARTITION RANGE SINGLE: *A coordinator step that confirms only partition 25 of KEYWORD is in scope (Pstart=25, Pstop=25). No data movement — purely a partition scoping signal to steps 5 and 6.*

Step 9 (executes fourth) — INDEX RANGE SCAN on IDX_DK_KEYWORD (all 34 partitions): *For each* `keyword_key` *from step 5, Oracle scans the* `doc_keyword` *bridge table index across all 34 reference partitions (Pstart=1, Pstop=34). Returns ROWIDs and* `document_key` *values for matching rows.*

Step 8 (executes fifth) — TABLE ACCESS BY LOCAL INDEX ROWID BATCHED on DOC_KEYWORD: *Fetches* `doc_keyword` *rows using the ROWIDs from step 9. Returns* `document_key` *values to drive the next join.*

Step 7 (executes sixth) — PARTITION REFERENCE ALL: *Scoping signal for the* `doc_keyword` *reference partition scan. Because* `doc_keyword` *is reference-partitioned off* `document` *and no document-side predicate exists, all 34 partitions must be considered.*

Step 3 (executes seventh) — NESTED LOOPS (keyword → doc_keyword): *The first join operator. For each keyword row from steps 4–5, it drives the doc_keyword lookup at steps 7–9. Outputs* `(keyword_key, document_key)` *pairs to the next operator.*

Step 10 (executes eighth) — INDEX UNIQUE SCAN on PK_DOCUMENT: *For each* `document_key` *from step 3, Oracle performs a unique lookup against the document primary key index to locate the document ROWID. Cost is 0 per row — this is a pure B-tree point lookup.*

Step 2 (executes ninth) — NESTED LOOPS (doc_keyword → document ROWID): *The second join operator. For each* `document_key` *from step 3, it drives the PK lookup at step 10. Passes the document ROWID up to step 11.*

Step 11 (executes tenth) — TABLE ACCESS BY GLOBAL INDEX ROWID on DOCUMENT: *Fetches the actual DOCUMENT row using the ROWID from step 10. The* `ABSTRACT IS NOT`

`NULL` filter is applied here — it cannot be pushed earlier because `abstract` is a CLOB stored out-of-row and cannot be evaluated at the index level. The CLOB is then passed to `DBMS_LOB.SUBSTR` in the SELECT list. Access method is `GLOBAL INDEX ROWID` because `PK_DOCUMENT` is unpartitioned; Oracle resolves the partition at runtime from the ROWID, shown as `Pstart=ROWID, Pstop=ROWID`.

Step 1 (executes eleventh) — NESTED LOOPS (document ROWID → document row): *The outermost join operator, linking the ROWID lookup at step 10 to the table fetch at step 11.*

Step 0 (executes last) — SELECT STATEMENT: *Returns the CLOB substrings to the client.*

SCIKEY Execution Plans — update this document whenever MV definitions, indexes, or refresh schedules change, as plan hashes will differ after structural changes.